

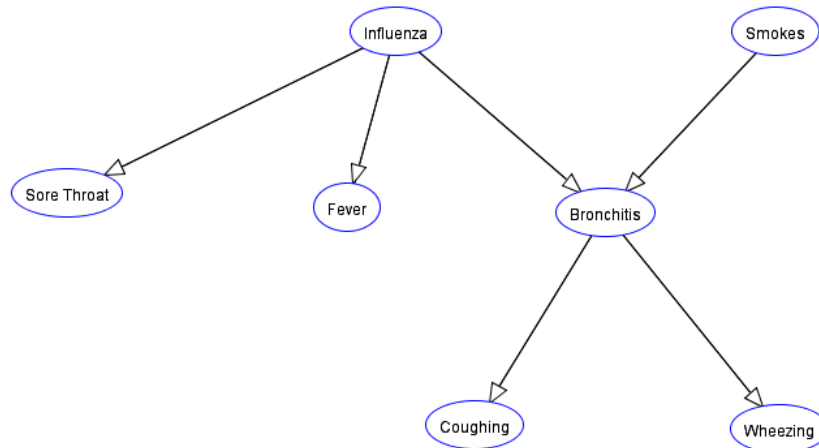
COMS W4701: Artificial Intelligence, Summer 2026

Homework 4

Instructions: Prepare submissions **only** for problems marked “Graded”. Compile written responses in a single, typed PDF file. Coding solutions may be directly implemented in the provided Python file(s). Do not modify any filenames or code outside of the indicated sections. Submit all required files on Pensive in the appropriate assignment bins. Please be mindful of the deadline, as well as our policies on citations and academic honesty.

[No Submission] Problem 1: Diagnostic Bayes Net

The following Bayes net is the “Simple Diagnostic Example” from the Sample Problems of the Belief and Decision Networks tool on AIspace. All variables are binary.



1. We want to compute the distribution $\Pr(\text{Wheezing} \mid \text{Fever} = \text{True})$. Find a minimal analytical expression for it (or its unnormalized version) in terms of the Bayes net CPTs. What is the maximum size of the intermediate factor if all marginalization is done at the end?
2. We employ variable elimination to solve the query above. Consider the following two variable orderings (you may only need to sum over a subset of each):
 - (i) influenza, smokes, sore throat, fever, bronchitis, coughing, wheezing
 - (ii) wheezing, coughing, bronchitis, fever, sore throat, smokes, influenza

For each of these orderings, rewrite your analytical expression by splitting up the sum over each variable, and factor out the maximal number of terms from each summation. What is the size of the largest intermediate factor in each?

3. Repeat both parts for the query $\Pr(\text{Sore Throat} \mid \text{Coughing} = \text{True})$.

[No Submission] Problem 2: Wandering Robot

A robot is wandering around a room with some obstacles, labeled as # in the grid below. It can occupy any of the free cells labeled with a letter, but we are uncertain about its true location and so we keep a belief distribution over its current location. At each timestep it moves from its current cell to a neighboring free cell in one of the four cardinal directions with uniform probability; it cannot stay in the same cell. For example, from A the robot can move to either B or C with probability $\frac{1}{2}$, while from D it can move to B, C, E, or F, each with probability $\frac{1}{4}$.

A	B	#
C	D	E
#	F	#

The robot also makes an observation after each transition, returning what it sees in a random cardinal direction. Possibilities include observing #, “wall”, or “empty” (for a free cell). For example, in B the robot observes “wall”, # (each with probability $\frac{1}{4}$), or “empty” (probability $\frac{1}{2}$).

1. Write out the transition matrix, following the ordering of the states from A to F. Also write out the observation probability vectors $O_{\#}$, O_{wall} , O_{empty} .
2. Let $\Pr(X_0)$ be a uniform distribution over all free cells. Starting from this belief state, the robot makes one transition and observation $e_1 = \text{“empty”}$, followed by a second transition and observation $e_2 = \#$. What are the belief distributions $\Pr(X_1 | e_1)$ and $\Pr(X_2 | e_1, e_2)$?
3. Compute the joint distribution $\Pr(X_1, X_2 | e_1, e_2)$. You can either write the result as a matrix A , where $A_{ij} = \Pr(X_1 = i, X_2 = j | e_1, e_2)$, or just list the joint probability values that are nonzero. What is the most likely *sequence(s)* of states over the two timesteps?
4. Now suppose we perform localization using a particle filter instead. We initialize the filter with six particles, one in each free cell, and perform the transition step. Identify a resultant particle distribution in which a *maximal* number of cells are now-particle free. Identify a different distribution in which a *minimal* number of cells are now particle-free.
5. Suppose we have two particles in state A, one in B, one in D, and two in F. For the observation of “wall”, compute the set of weights that would be applied to a particle in these states. Also compute the *distribution* from which we would resample the particles.

[No Submission] Problem 3: Decision Tree

Consider the following data set. There are two trinary features F_1 and F_2 , and a class variable Y with values 0 and 1.

Sample	F_1	F_2	Y
$\mathbf{x}_1$	+1	-1	0
$\mathbf{x}_2$	+1	+1	0
$\mathbf{x}_3$	0	0	0
$\mathbf{x}_4$	0	-1	1
$\mathbf{x}_5$	0	+1	1
$\mathbf{x}_6$	-1	+1	1

1. We train a decision tree on the given data. Compute the average Gini index that results from splitting on each of the two features at the root node. Which feature gets chosen? Show how the 6 samples are split according to the chosen feature.
2. Repeat the splitting process for any bins from above where the training data are not pure. Show how the samples are split according to it.
3. Sketch the learned decision tree. Clearly label the feature at each non-leaf node and the feature value along the branches. For leaf nodes, clearly label the predicted value of Y . If the decision is indeterminate (e.g., due to no data or an equal proportion of data in a leaf node), you may indicate as such.

[Graded] Problem 4: Grid World Localization

You will be implementing grid world localization (see Section 14.3.2 in AIMA for a similar example). An agent is moving around and gathering observations on a grid, where each cell may be either passable or not passable (e.g., a wall). We do not know exactly where the agent is, so we will have a belief state over each cell of the grid. For simplicity, the belief state will also include the blocked cells, which will always have probability 0. You will be completing the `Gridworld_HMM` class to implement the environment model, as well as the forward and particle filter algorithms.

We may test your code on different grid worlds. Please do not hardcode any information about the specific example shown above, including sizes and wall locations.

4.1: Transition Probabilities (10 points)

The transition model is such that the agent may either stay in the current cell or move to an adjacent free cell, all with uniform probability. Thus, if the set S includes all adjacent free neighbors of state x in addition to x itself, then $\Pr(x'|x) = \frac{1}{|S|}$ for all $x' \in S$.

Write the function `initT()`, which should return a $N \times N$ array T , where N is equal to `grid.size` and $T_{ij} = \Pr(x_j|x_i)$. We recommend that you populate T one row (state) at a time, following the cell order enumerated in the figure. You may use the `neighbors()` helper function, which returns a list of all adjacent free cells and the given cell. Be sure to check that the rows of T sum to 1.

4.2: Observation Probabilities (10 points)

From a state x , the agent can make an observation e , which is an integer value between 0 and 15 (inclusive). The 4-bit representation of e indicates whether each of the four adjacent neighbors is a blocked (1) or free (0) cell. The order of the bits is north, east, south, west (NESW). For example, the correct observation for the lower-right corner cell in the figure would be the value 6, with bit

representation 0110 indicating free, blocked, blocked, free cells in NESW order.

However, the observations are also *noisy*; there is a ϵ probability that each bit is independently wrong. Let e^* be the correct observation for state x , e be the actual observation, and d be the number of mismatched bits between e and e^* . Then $\Pr(e|x) = (1 - \epsilon)^{4-d} \epsilon^d$. Following the example above, $\Pr(e = 6|x) = (1 - \epsilon)^4$, while $\Pr(e = 0|x) = \Pr(e = 15|x) = (1 - \epsilon)^2 \epsilon^2$.

Write the function `initO()`, which should return a $16 \times N$ array O where $O_{ij} = \Pr(e = i|x_j)$. Again, it may be easiest to populate the array one column (state) at a time. First find the “correct” observation value for each state. Then for each possible observation value between 0 and 15, compute the discrepancy¹ and insert the observation probability into the array.

4.3: Forward Algorithm (6 points)

Now that we have the model defined, we can perform inference. `forward()` is given a list of `observations` and the initial belief state `init`. It should return a $T \times N$ NumPy array, where T is the number of observations and N is equal to `grid.size`, containing the *normalized* $\Pr(X_t|e_{1:t})$ belief states in each row.

4.4: Particle Filter (12 points)

Alternatively, we can use a particle filter to perform approximate inference. We will store the set of particles in an array `self.particles`, such that `self.particles[i] = s` indicates that particle i is in state s . We will also have an identically sized array storing weight values of each particle.

The filter will be implemented via three helper and one main function. The `transition()` function should sample a successor state for each particle according to the transition matrix, and update `self.particles` in place². The `observe()` function should compute the weights for each particle given an observation value, and update `self.weights` in place. Finally, the `resample()` function should generate a new set of particles using the current set of weights. This should update `self.particles` in place, and also reset all `self.weights` to 1.

You will then put everything together in `particle_filter()`. Like `forward()`, it will perform inference by executing the three procedures above for each individual observation. It should also return a $T \times N$ array, in which each row contains the *counts* of particles in each state (think of this as an approximation of the belief distribution).

4.5: Analysis (12 points)

When you are finished, you can test your implementation by invoking `python main.py`. It can run under four “modes”, specified by the argument `-m`, as described below:

- Mode 0 (default): Run the forward algorithm for `-t` timesteps (default 50), `-n` episodes (default 100), and a set of ϵ values. Five different values of ϵ (0.4, 0.2, 0.1, 0.05, 0.02) are tested. When finished, a plot showing the average localization error³ will be shown.

¹You can do so in Python using `bin(x^y).count('1')`.

²`np.random.choice()` should be helpful here.

³This is computed as the total Manhattan distance between the agent’s belief distribution and its true location (a distribution with a value of 1 in its true location and 0 elsewhere).

- Mode 1: Run the particle filter for `-t` timesteps, `-n` episodes, a set of ϵ values, and `-p` (default 30) particles. When finished, a plot showing the average localization error will be shown.
- Mode 2: Run the forward algorithm for `-t` timesteps with ϵ equal to `-e` (default 0.02). An animation will pop up showing the agent's true location and the estimated belief distribution colored on the grid over time. Bright yellow corresponds to higher probabilities.
- Mode 3: Run the particle filter for `-t` timesteps with ϵ equal to `-e` and `-p` particles. An animation similar to that in Mode 2 will pop up here. There will also be values showing the number of particles on certain cells (cells without values have 0 particles).

After verifying that your implementation is working, briefly address the following questions.

1. Show the error plot for Mode 0 in your writeup. How does the value of ϵ affect the performance of the forward algorithm?
2. Show the error plot for Mode 1. How does the performance of the particle filter generally compare to exact inference using the forward algorithm?
3. Run Mode 3 a few times—you should see both successes and failures when using the particle filter. Briefly explain why tracking failures occur even when the particles are relatively “confident” about the agent's location.

Submission

Please submit the `gridworld_hmm.py` file on Pensive.